

Scaling Multi-Turn RAG: High-Performance Parallelized Pipeline for the MTRAG Benchmark

Murat Çelik Nejla Dinçer Can Ersoy

Mert Toprak Barış Tan Ünal

Politecnico di Torino

Corso Duca degli Abruzzi, 24 10129 Torino, Italy

{s338212, s343290, s336887, s336966, s338210}@studenti.polito.it

Abstract

Recently, Retrieval-Augmented Generation (RAG) has become a significant task in Large Language Models (LLMs). In multi-turn Retrieval-Augmented Generation (RAG), a good system must overcome the challenges of maintaining context as the dialogue turns progress and manage the issue of generating answers depending on conversation history. Our project presents a high-performance, parallelized Multi Turn RAG pipeline designed for systematic evaluation of the MTRAG benchmark across its three subtasks: Retrieval (Task A), Generation (Task B), and End-to-End RAG (Task C). Our methodology utilizes a Streamlit framework that allows users to embed diverse corpora with varying vector spaces and embedding models, facilitating configuration for each task regarding the task's nature. Some key experiments are focused on the performances of different vector databases, embedding models, the necessity of LLM-based query rewriting for non-standalone questions, utilization of different rerankers and scale and performance of the selected LLM for the answer generation. We conclude that a configuration utilizing query rewriting along with reranking delivers the better results comparing to their absence. You can find our GitHub repository [here](#).

1 Introduction

Large Language Models (LLMs) have revolutionized information-seeking tasks, yet they often struggle with context preservation, hallucination and generating answers given a corpora. Retrieval-Augmented Generation (RAG) addresses this by basing the model responses on relevant external passages. While single-turn RAG is becoming a immerse field, Multi-Turn RAG presents a more complex challenge where the system must generate responses to a question within the context of an ongoing conversation. Unlike single-turn, multi-turn interactions require the LLM to track conversational history and evolve its retrieval strategy as

the dialogue progresses. One of the main causes of the challenge is because of the non-standalone nature of human dialogue. In later turns of a conversation, users frequently use shortcuts, such as referencing (e.g., "its address?" or "the previous one"), which make the current question ambiguous without the preceding context. Moreover, the system requires active retrieval strategies, where it must decide whether a question is answerable based on the available corpus it is provided with or if it should decline to answer to avoid hallucinations.

The goal of our project is to develop a modular, parallelized RAG pipeline specifically optimized for the MTRAG benchmark. Our system is designed to efficiently process 842 distinct tasks across four diverse domains: CLAPNQ, FIQA, Govt, Cloud. By leveraging a Streamlit-based framework, we aim to maximize performance metrics such as nDCG and Recall for retrieval and RBalg, RBllm and RLf for generation. This architecture allows for a systematic exploration of the performance comparison for the following based on the aforementioned metrics: Varying the utilization of different embedding models, vector databases, existence of query rewriting, existence and utilization of reranking with different top-k values and use of different LLMs for generation. The MTRAG Benchmark contains three subtasks:

Task A (Retrieval): Evaluating the system's ability to retrieve gold-standard passages for the final turn of a conversation.

Task B (Generation): Assessing the generator's ability to produce a faithful answer when provided with perfect reference passages.

Task C (End-to-End RAG): Evaluating the complete pipeline, where the system must perform its own retrieval before generating a final response for the user's last question.

Our work focuses on providing a system such that the user can dynamically try different configu-

ration settings so that they can receive better metric results. We have also provided an evaluation section such that it is easily possible to visualize the outcomes of their prediction results for each task. We have tested and evaluated the MTRAG Benchmark dataset with various configurations, which will be analyzed in depth in the following sections.

2 Related Work

2.1 Multi-Turn RAG Evaluation

Recent advancements in Large Language Models (LLMs) have necessitated robust evaluation frameworks. While benchmarks like MT-Bench [Zheng et al. \(2023\)](#) evaluate multi-turn conversation capabilities using LLM-as-a-judge, they often lack a specific focus on Retrieval-Augmented Generation (RAG) challenges such as domain-specific retrieval and unanswerable questions. The MTRAG Benchmark [Katsis et al. \(2025\)](#) addresses this gap by stressing the system’s ability to handle unanswerable queries. Unlike static, single-turn datasets like Natural Questions or MS-MARCO where an answer is always guaranteed, MTRAG requires the model to discern when retrieved contexts are insufficient and explicitly abstain from answering, thereby testing its robustness against hallucination.

2.2 Retrieval Strategies

The retrieval component is the backbone of any RAG system. Traditional Sparse Retrieval methods, such as BM25 [Robertson and Zaragoza \(2009\)](#), rely on exact keyword matching. While efficient, they suffer from the lexical gap problem, failing to capture semantic synonyms. Conversely, Dense Retrieval [Karpukhin et al. \(2020\)](#) employs dual-encoder architectures to map queries and documents into a shared latent space, capturing semantic similarity. Modern state-of-the-art pipelines often employ a hybrid approach or a two-stage "Retrieve-and-Rerank" architecture. In this setup, a lightweight retriever fetches a broad set of candidates, which are then re-scored by a Cross-Encoder [Nogueira and Cho \(2019\)](#) to maximize precision at the cost of higher latency.

2.3 Query Rewriting in Conversational AI

A unique challenge in multi-turn RAG is the ambiguity of user queries due to coreferences (e.g., "how much is it?" referring to a previously mentioned product). Direct retrieval using such raw queries often leads to poor performance. Query Rewriting

(QR) addresses this by generating a standalone, de-contextualized query using the conversation history [Vakulenko et al. \(2021\)](#). This rewritten query effectively bridges the gap between conversational nuances and the need for explicit search terms in vector databases.

3 Dataset

The MTRAG benchmark aggregates data from four diverse domains to ensure robust evaluation across different linguistic styles and knowledge types:

- 1. ClapNQ:** Adapted from the Natural Questions dataset, this domain focuses on open-domain factoid questions based on Wikipedia articles. It serves as a general-knowledge baseline.
- 2. Cloud:** Comprising technical documentation from major cloud service providers. This domain tests the system’s ability to handle jargon-heavy, structured technical content.
- 3. FiQA:** Financial dataset from forums and reports. It requires numerical reasoning and specialized terminology.
- 4. Govt:** Sourced from various government information portals. This domain involves parsing bureaucratic language and complex policy details.

4 Methodology

Our system adopts a modular, component-based architecture designed for flexibility and high performance. The core pipeline consists of sequential stages:

- 1. User Input Processing:** The system accepts a file containing a list of objects, each comprising a raw user query along with the conversation history.
- 2. Query Rewriting:** An LLM-based rewriter transforms the context-dependent query into a standalone search query.
- 3. Embedding Generation:** The rewritten query is converted into a dense vector embedding.
- 4. Vector Retrieval:** A high-speed vector database (Qdrant or Faiss) retrieves the top- K most similar document chunks.
- 5. Reranking:** A cross-encoder model re-scores the retrieved candidates to improve precision, filtering down to the top- N context passages.
- 6. Generation:** The final context is fed into the LLM generator to produce the answer.

4.1 Task A: Retrieval Only

Task A isolates the retrieval subsystem. Given a user query and conversation history, the system

must retrieve the specific passages that contain the answer. **Query Rewriting (QR):** A critical challenge in multi-turn conversations is detecting the intent of elliptical or coreferential queries (e.g., "Why did he do that?"). We employ a generative Query Rewriter that leverages the conversation history to resolve these ambiguities. The rewriter is prompted to produce a fully specified, standalone question that captures the user’s original intent without requiring external context, which is essential for effective vector similarity search.

Vector Search Infrastructure: We evaluated two distinct vector search backends:

Faiss: A library for efficient similarity search and clustering of dense vectors. It provides a lightweight, in-memory solution suitable for rapid prototyping but lacks advanced filtering features.

Qdrant: A production-grade vector database that supports HNSW (Hierarchical Navigable Small World) indexing for approximate nearest neighbor search. Crucially, Qdrant allows for payload-based filtering, enabling efficient retrieval within specific subsets of the corpus.

Two-Stage Retrieval & Reranking: To balance recall and precision, we implement a two-stage retrieval process. In the first stage, the vector database retrieves a broad set of candidates (e.g., $\text{Top-}K = 20$) based on cosine similarity. In the second stage, a Cross-Encoder model (such as BGE-Reranker or MS-MARCO) performs a computationally intensive re-scoring of these candidates. This reranking step is vital for filtering out "hard negatives" (documents that are semantically similar but factually irrelevant) thereby presenting the generator with only the highest-quality context ($\text{Top-}N = 10$).

4.2 Task B: Generation with Reference Passages

For Task B generation, our main goal is to generate relevant answers to the last question in the given set of tasks. Task is defined as a conversation turn containing all previous turns together with the last user question (e.g., the task created for turn k includes all user and agent questions/responses for the first $k - 1$ turns plus the user question for turn k).

To start generation process, the system converts the MTRAG input format into LangChain-compatible message objects. Then, all turns except the last one are converted to HumanMessage(user) or AIMessage(agent) objects and stored as conver-

sation history. Subsequently, the last user turn is extracted as the current question to be answered.

Context Management: Retrieved passages are formatted into a structured context string. Each passage is numbered sequentially and presented in order of the relevance score(descending order). Afterwards, the system constructs a comprehensive prompt concatenating 4 key components: **System Prompt:** Defines the assistant’s(LLM model) role and instructions.(See appendix B.3) **Conversation History:** All previous turns (HumanMessage/AIMessage pairs). **Current Question:** The latest user query. **Context:** Reference passages for generation. After assembling all the parts for the prompt, it is then sent to the selected LLM model to generate an answer for the user’s question.

Model Selection: We used a variety of LLM models to compare which give more reasonable answers. We selected models from different companies and with different sizes to see variations in the generation results between larger/smaller models and different vendors. The models we used for generation are namely; gpt-oss-20b from openai, gemma-3-12b from google, deepseek-r1-0528-qwen3-8b from deepseek, ministral-3-14b-reasoning and ministral-3-3b from mistralai.

4.3 Task C: Generation with Retrieved Passages

Task C combines retrieval and generation in a single pipeline. Given a conversation and a document corpus, the system first retrieves relevant passages and then generates an answer based only on those passages.

Query Rewriting: The last user question may contain unclear references or missing details. We use a Query Rewriter that looks at the full conversation and rewrites the question into a clear, standalone form. This helps improve retrieval accuracy.

Passage Retrieval: The rewritten query is converted into a vector and searched against the document corpus using a vector database (Qdrant). We retrieve the $\text{Top-}K = 20$ most similar passages to ensure high recall.

Reranking and Selection: A cross-encoder reranker rescales the relevance of the retrieved passages. Based on these scores, we select the $\text{Top-}N$ passages. We return at most $N = 5$ or $N = 10$ passages, which provides enough context while reducing noise.

Context Building: The selected passages are

ordered by relevance and combined into a single context block. This context is given to the generation model.

Answer Generation: The final prompt is formed by combining system instructions, conversation history, retrieved context, and the current user question, and is then passed to the LLM to generate the final answer.

This prompt is sent to the LLM, which generates the final answer using only the retrieved passages, ensuring factual and faithful responses.

5 Experimental Results

All experiments were conducted on a heterogeneous compute cluster utilizing Apple M3 Pro silicon for local inference and NVIDIA T4 GPUs for heavier reranking tasks. We evaluated the retrieval pipeline (Task A) using **Recall@K** and **nDCG@K** to measure context relevance and ranking quality. For generation quality (Tasks B & C), we employed an LLM-as-a-Judge to compute two critical metrics: **Faithfulness** (RL_F), a binary score indicating whether the answer is derived *solely* from the provided context to prevent hallucinations; and **IDK Accuracy**, which measures the model’s ability to correctly refuse to answer when retrieved context is insufficient.

To isolate the contribution of each component, we defined a hyperparameter search space across three dimensions. For **Embeddings**, we compared `all-minilm:l6-v2` (efficient), `nommic-embed-text` (long-context), and `jina-embeddings-v2` (high-performance). For **Reranking**, we evaluated `BGE-v2-M3`, `Mxbai-Rerank-xsmall`, and `cross-encoder/ms-marco`. Finally, we tested varying **Retrieval Depths**, specifically identifying the Top- K ($K = 20, 30$) candidates from the vector store and reranking the Top- N ($N = 5, 10$) for final context generation.

5.1 Task A: Retrieval Performance

We evaluated the retrieval pipeline based on Normalized Discounted Cumulative Gain ($nDCG@k$) and Recall ($R@k$) at varying depths ($k = 3, 5$). Our primary objective was to maximize the relevance of context provided to the generation phase while maintaining low latency. We established a **Baseline** configuration using Qdrant, `all-minilm-l6-v2` embeddings, and no reranking, which yielded an $nDCG@3$ of 0.249.

5.1.1 Query Rewriting Strategies

Given the conversational nature of MTRAG, where queries often depend on previous turns (e.g., “What about in 2024?”), direct retrieval often fails. We compared two models for Query Rewriting (QR): `google/gemma-3-12b` and `openai/gpt-oss-20b`. While enabling QR with the smaller Gemma model improved the baseline $nDCG@3$ to 0.268 (+7.6%), the larger GPT-20b model achieved 0.293 (+17.6%). This substantial gap indicates that the reasoning capabilities required to disambiguate complex multi-turn references scale significantly with model size. Consequently, we utilized GPT-20b for all subsequent experiments to isolate downstream architectural effects.

5.1.2 Impact of Reranking Architectures

To mitigate the precision limitations of dense vector retrieval, we introduced a second-stage reranking step. We experimented with cross-encoder architectures to re-score the top- k documents retrieved by the vector store.

Reranker Model	Depth	nDCG@3	R@3
<i>None (Baseline)</i>	-	0.293	0.275
MS-Marco-MiniLM	10	0.293	0.275
Mxbai-xsmall-v1	10	0.317	0.300
BGE-v2-M3	10	0.329	0.311
BGE-v2-M3	5	0.342	0.326

Table 1: Comparison of Reranking models and depths. All experiments used Qdrant, `all-minilm-l6-v2` embeddings, and GPT-20b for Query Rewriting.

As shown in Table 1, modern rerankers significantly outperform older architectures. While MS-Marco provided negligible gains over the non-reranked pipeline, BGE-v2-M3 achieved the highest performance across all metrics. We further investigated the “re-ranking depth”—the number of documents passed to the reranker. Counter-intuitively, reranking only the top-5 documents yielded higher performance ($nDCG@3$: 0.342) than reranking the top-10 ($nDCG@3$: 0.329). This suggests that for this specific benchmark, the documents ranked 6–10 by the vector store act as “distractors.” Expanding the window introduces more noise than signal, causing the reranker to falsely promote irrelevant passages.

5.1.3 Embedding Models and Infrastructure

We evaluated the semantic capture capabilities of four embedding models. As shown in Table 2, larger models do not strictly guarantee better per-

formance on the MTRAG dataset. The state-of-the-art jina-embeddings-v2 achieved the highest accuracy (0.332), but the compact all-minilm model remained extremely competitive (0.329). Given the negligible performance difference ($< 1\%$), all-minilm represents a superior efficiency-performance trade-off for latency-sensitive applications. Conversely, snowflake-arctic suffered a catastrophic drop (0.059), likely due to domain mismatch or tokenization issues.

Embedding Model	nDCG@3	R@3
Snowflake-Arctic (137m)	0.059	0.058
Nomic-Embed-Text	0.227	0.219
All-MiniLM-L6-v2	0.329	0.311
Jina-Embeddings-v2	0.332	0.313

Table 2: Impact of Embedding Models on retrieval performance. Experiments utilized Qdrant and BGE-v2-M3 (Top-10) reranking.

Additionally, we performed an infrastructure sanity check by comparing Qdrant against FAISS. Using identical parameters (Nomic embeddings, BGE reranker), FAISS yielded significantly lower accuracy (nDCG@3: 0.050) compared to Qdrant (0.227), highlighting the critical impact of vector store configuration (e.g., quantization, distance metrics) on end-to-end performance.

5.2 Task B: Generation Performance

Model Comparison: Here, we present the results of our Task B evaluation inside Table A.2. General trend about overall performance of the models can be clearly seen, as the model size increase, the performance also increases. However, it is not always the case. gemma-3-12b is the best performing model out of 5 models in comparison based on the harmonic mean of $\mathbf{RL}_F$, $\mathbf{RB}_{llm}$ and $\mathbf{RB}_{alg}$ scores.

gpt-oss-20b is the second best performing model following gemma-3-12b. Even though gpt-oss-20b has a larger size (20B), it is falling behind gemma-3-12b in nearly all metrics, particularly in faithfulness (0.82 vs 0.74). This may be due to several architectural factors: Gemma-3 utilizes a 5-to-1 interleaved attention mechanism which reduces KV-cache memory pressure compared to standard dense architectures like GPT-oss, allowing the model to maintain higher precision across the long contexts required in multi-turn RAG. Another reason can be the difference between distillation and pretraining quality of models. Gemma likely received extensive instruction tuning

on document grounded generation which helps it stay more compliant with referenced documents compared to GPT-oss. Additionally, it has been noted in community benchmarks about GPT-oss to have higher hallucination rates. This is mostly related to its relatively larger size, allowing model to make more creative, fluent, and well-structured responses.

ministral-3-3b is particularly standing out ($\mathbf{H.mean}=0.51$ and $\mathbf{RL}_F=0.79$), among significantly larger 8B and 20B models despite its smaller size with its performance, which is achieved by cascade distillation from a 24B parameter parent (Mistral Small 3.1) with high level reasoning capabilities, combined with tied input-output embeddings that maximize parameter allocation. Furthermore, its use of Sliding Window Attention ensures consistent computational costs and prevents the context degradation typically seen in smaller models in multi-turn scenarios, allowing it to perform better in extended dialogues.

Hallucination Analysis: When analyzing which models stay truest to the facts, Gemma-3-12b comes as the first ($\mathbf{RL}_F=0.82$). It is particularly good at setting aside its prior training to focus strictly on the information provided in the documents. In contrast, GPT-oss-20b has a knowledge leakage problem which makes it rely too heavily on its own internal memory; while it writes very smoothly, it often ignores the provided evidence in favor of what it already knows. Meanwhile, reasoning models like DeepSeek-R1 suffer from over-reasoning the answer, inventing some steps that aren’t actually supported by the text. Interestingly, the compact Ministral-3-3b shows that is not necessary to have a massive model to be accurate ($\mathbf{RL}_F=0.76$); its smaller size limits its ability to imagine false details, forcing it to stick closer to the provided context.

5.3 Task C: End-to-End Performance

In Task C the generator must rely solely on the noisy context provided by the retrieval system. Table 3 presents the performance of our top configurations. While running settings for this part, we experimented with the best results of Task A and Task B, since Task C is a combination of them.

Similar to Task B, we obtained the best ranking with the gemma-3-12b model which outperformed the larger, gpt-oss-20b. This highlights the efficiency of the gemma-3 instruction-tuning for RAG tasks; while gpt-oss-20b achieved strong

Ranking	LLM Model	Embedding Model	Reranker	TopK	H.Mean	RL _F	RB _{llm}	RB _{alg}
1st	Gemma-3-12b	Jina-Embed-v2	BGE-Reranker-v2	10	0.53	0.84	0.63	0.34
1st	Gemma-3-12b	All-MiniLM-L6	BGE-Reranker-v2	5	0.52	0.85	0.59	0.35
2nd	Gpt-oss-20b	All-MiniLM-L6	BGE-Reranker-v2	10	0.50	0.81	0.67	0.30
3rd	Gpt-oss-20b	Jina-Embed-v2	BGE-Reranker-v2	10	0.49	0.77	0.65	0.30
4th	Gpt-oss-20b	All-MiniLM-L6	Mxbai-XS	10	0.48	0.79	0.65	0.29
5th	Gpt-oss-20b	All-MiniLM-L6	BGE-Reranker-v2	5	0.48	0.76	0.61	0.30
6th	Gpt-oss-20b	Nomic-Embed	BGE-Reranker-v2	10	0.39	0.62	0.50	0.24

Table 3: Task C performance comparison according to different settings, ranking based on Harmonic Mean, calculated using **RL_F**, **RB_{llm}**, **RB_{alg}** metrics.

results with a larger context window, gemma-3-12b demonstrated superior adherence to the retrieved context with a significantly higher **RL_F**.

The results underscore the critical dependence of generation quality on the underlying retrieval stack, which is discussed in detail in Section 5.1. We observed significant performance variances based on the embedding and reranking models used. **Reranker Importance:** The inclusion of **BGE-Reranker-v2** consistently yielded higher scores compared to others. In **Embedding Sensitivity** context, **All-MiniLM-L6** performed better in general although the best case was with **Jina-Embed-v2**.

Due to the nature of Task C, retrieval failures propagate to the generation phase. This is particularly evident when comparing the distributions of the evaluation metrics:

Low Algorithmic Overlap (RB_{alg}): Across all runs, **RB_{alg}** scores are notably low (0.24–0.35). This metric relies on lexical overlap with the reference answer. In Task C, if the retriever fails to find the exact passage used to write the reference answer, the generator will produce a response that diverges lexically from the reference.

High Faithfulness (RL_F): Conversely, **RL_F** scores remain relatively high (up to 0.85). This indicates that the models are generally faithful to the retrieved context, even if that context is incorrect or irrelevant.

This disparity suggests that the bottleneck in our end-to-end performance is primarily the retrieval recall rather than the generation capability; the models are not hallucinating wildly, but are often answering the wrong specific details because the retrieval step failed to provide the gold passages.

5.4 Discussion

The experimental results reveal several insights for MTRAG systems. For retrieval part, **Query Rewriting** performance scales significantly with model size. While a smaller model like gemma-3-

12b improved the retrieval baseline by 7.6%, the larger gpt-oss-20b model achieved a 17.6% improvement. This indicates that resolving complex coreferences in human dialogue requires the higher reasoning capabilities of larger LLMs. **Reranking Depth** is another factor as reranking only the top-5 documents yielded better performance in general than reranking the top-10. This suggests that lower-ranked documents often act as "distractors," introducing more noise than signal into the reranker.

As for generation phase, it is observed that **generator faithfulness** is not strictly correlated with model size. gemma-3-12b outperformed the larger gpt-oss-20b in adherence to retrieved context (**RL_F** of 0.82 vs 0.74). This is attributed to architectural advantages like interleaved attention and specific instruction tuning for document-grounded generation as explained thoroughly in Section 5.2.

6 Conclusion

We presented a high-performance, modular RAG pipeline designed for the MTRAG benchmark. Our system evaluates three distinct subtasks: Retrieval (Task A), Generation (Task B), and End-to-End RAG (Task C). Our work demonstrates that Multi-Turn RAG requires robust LLM-based Query Rewriting to resolve ambiguities and an optimized two-stage "Retrieve-and-Rerank" architecture to maximize precision. Our experiments also reveal that smaller, state-of-the-art models like gemma-3-12b can outperform larger baselines like gpt-oss-20b. We attribute this to superior architectural efficiency (specifically interleaved attention mechanisms) which allow for more robust context tracking in multi-turn scenarios compared to the standard sparse patterns of larger models. Future research will focus on handling "unanswerable" questions to reduce hallucination rates. Additionally, we plan to explore synthetic data augmentation using the MTRAG-S framework to improve the system's robustness across diverse domains.

References

- Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781.
- Yannis Katsis, Sara Rosenthal, Kshitij Fadnis, Chulaka Gunasekara, Young-Suk Lee, Lucian Popa, Vraj Shah, Huaiyu Zhu, Danish Contractor, and Marina Danilevsky. 2025. [Mtrag: A multi-turn conversational benchmark for evaluating retrieval-augmented generation systems](#).
- Rodrigo Nogueira and Kyunghyun Cho. 2019. Passage re-ranking with bert. *arXiv preprint arXiv:1901.04085*.
- Stephen Robertson and Hugo Zaragoza. 2009. [The probabilistic relevance framework: Bm25 and beyond](#). *Foundations and Trends in Information Retrieval*, 3:333–389.
- Svitlana Vakulenko, Shayne Longpre, Zhucheng Tu, and Raviteja Anantha. 2021. Question rewriting for conversational question answering. In *Proceedings of the 14th ACM International Conference on Web Search and Data Mining*, pages 355–363.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Sanskar Aketi, Mayur J. Rice, Bei Chen, Dacheng Tao, Gunho Park, Ion Stoica, Michael I. Jordan, and Xuanzhe Liu. 2023. [Judging llm-as-a-judge with mt-bench and chatbot arena](#). *Preprint*, arXiv:2306.05685.

Appendix

A Detailed Retrieval and Generation Experiments

ID	QR	VectorDB	QR Model	Embedding	Reranker	Depth	nDCG@1	nDCG@3	nDCG@5	nDCG@10	R@1	R@3	R@5	R@10
1	YES	Qdrant	GPT-oss-20b	All-MiniLM-L6	BGE-v2-M3	5	0.360	0.342	0.384	0.382	0.149	0.326	0.431	0.431
2	YES	Qdrant	GPT-oss-20b	Jina-Embed-v2	BGE-v2-M3	10	0.349	0.332	0.364	0.419	0.148	0.313	0.397	0.529
3	YES	Qdrant	GPT-oss-20b	All-MiniLM-L6	BGE-v2-M3	10	0.348	0.329	0.365	0.418	0.143	0.311	0.406	0.533
4	YES	Qdrant	GPT-oss-20b	All-MiniLM-L6	BGE-v2-M3	10	0.335	0.320	0.353	0.406	0.139	0.303	0.392	0.518
5	YES	Qdrant	GPT-oss-20b	All-MiniLM-L6	Mxbai-XS	10	0.337	0.317	0.351	0.403	0.139	0.300	0.389	0.511
6	YES	Qdrant	Gemma-3-12b	All-MiniLM-L6	BGE-v2-M3	5	0.326	0.317	0.359	0.357	0.133	0.302	0.405	0.405
7	YES	Qdrant	GPT-oss-20b	All-MiniLM-L6	Jina-Reranker-v2	10	0.334	0.316	0.350	0.407	0.140	0.300	0.388	0.522
8	YES	Qdrant	GPT-oss-20b	All-MiniLM-L6	MS-Marco	10	0.320	0.293	0.324	0.369	0.134	0.275	0.356	0.464
9	YES	Qdrant	GPT-oss-20b	All-MiniLM-L6	None	-	0.320	0.293	0.324	0.369	0.134	0.275	0.356	0.463
10	YES	Qdrant	Gemma-3-12b	All-MiniLM-L6	BGE-v2-M3	10	0.303	0.290	0.328	0.381	0.124	0.274	0.368	0.495
11	YES	Qdrant	Gemma-3-12b	All-MiniLM-L6	Mxbai-XS	10	0.308	0.286	0.326	0.376	0.126	0.268	0.366	0.483
12	NO	Qdrant	-	All-MiniLM-L6	BGE-v2-M3	10	0.285	0.269	0.300	0.346	0.120	0.255	0.334	0.443
13	YES	Qdrant	Gemma-3-12b	All-MiniLM-L6	None	-	0.284	0.268	0.295	0.344	0.117	0.253	0.324	0.438
14	NO	Qdrant	-	All-MiniLM-L6	None	-	0.262	0.249	0.269	0.308	0.113	0.237	0.293	0.385
15	YES	Qdrant	GPT-oss-20b	Nomic-Embed	BGE-v2-M3	10	0.234	0.227	0.256	0.301	0.099	0.219	0.292	0.398
16	YES	Qdrant	Gemma-3-12b	Nomic-Embed	BGE-v2-M3	10	0.232	0.230	0.253	0.297	0.095	0.224	0.285	0.389
17	NO	Qdrant	-	Nomic-Embed	BGE-v2-M3	10	0.227	0.210	0.233	0.273	0.100	0.198	0.256	0.350
18	YES	Qdrant	Gemma-3-12b	Nomic-Embed	None	-	0.224	0.207	0.230	0.264	0.092	0.197	0.257	0.339
19	YES	Qdrant	GPT-oss-20b	Nomic-Embed	None	-	0.216	0.205	0.230	0.267	0.091	0.197	0.260	0.350
20	NO	Qdrant	-	Nomic-Embed	None	-	0.211	0.188	0.208	0.240	0.094	0.174	0.227	0.300
21	YES	Qdrant	GPT-oss-20b	Snowflake-137m	BGE-v2-M3	10	0.058	0.059	0.067	0.083	0.024	0.058	0.078	0.114
22	YES	Faiss	GPT-oss-20b	Nomic-Embed	BGE-v2-M3	10	0.039	0.050	0.078	0.194	0.014	0.050	0.106	0.397

Table A.1: Full Experimental Results for Task A (Retrieval), sorted by nDCG@3 performance. Abbreviations: QR (Query Rewriting).

Rank	LLM Model	H.Mean	RL _F	RB _{llm}	RB _{alg}
1st	Gemma-3-12b	0.59	0.82	0.69	0.42
2nd	Gpt-oss-20b	0.55	0.74	0.74	0.36
3rd	Minstral-3-14b-reasoning	0.54	0.76	0.68	0.36
4th	Deepseek-r1-0528-qwen3-8b	0.52	0.75	0.66	0.35
5th	Minstral-3-3b	0.51	0.79	0.71	0.31

Table A.2: Task B generation performance including Faithfulness and Reference-Based metrics.

B Default System Prompt Used for Tasks B and C

You are an expert generative AI assistant in a RAG pipeline.

When given a user query: 1. Always prioritize *only information from the retrieved documents* included in the context. 2. Do not hallucinate facts or invent details not supported by those documents. 3. If the retrieved context lacks necessary information, acknowledge it clearly. 4. If there is no answer in the retrieved contents answer like this: I’m sorry, but I don’t have the answer to your question. 5. Follow these steps: a. Briefly summarize the relevant retrieved content linked to the query. b. Integrate that content into a coherent answer. c. Cite or reference the source document or snippet when appropriate. 6. Use precise, clear language and format answers for readability. 7. Do not write for the indicating like, answer, source and any others

Answer the user’s question now using the retrieved context.

Table B.3: Our default pre-prompt (before adding context, conversation history and user query) for generation tasks (For Task B and C. The prompt clearly explains the specific task to be performed, emphasizing on staying strictly compliant with the given context documents.